

Documento 5 — O Ciclo de Simulação

Autores: Yasmin Gomes de Matos e Ana Luiza Fernandes Novaes (Subgrupo D)

Data: 07/10/2026

1. Controle de Acesso e Isolamento de Simulações

1.1 Objetivo

O ecossistema Simbionte permite que múltiplos pesquisadores executem experimentos simultâneos. Para evitar interferências externas e impedir o consumo indevido da cota de requisições aos provedores de LLM, é fundamental garantir que o ciclo de vida de uma simulação seja estritamente governado pelo seu criador.

1.2 Regra e Implementação

Foi implementado um middleware de barreira (`verificarProprietario.ts`) na camada de roteamento da API. Antes de qualquer transição de estado que acione o motor de simulação (iniciar, pausar ou encerrar), o filtro intercepta a requisição e valida a identidade do usuário autenticado no token JWT (`user_id`) contra o proprietário registrado na simulação (`owner_id`).

* **Condição de aprovação:** `user_id == owner_id`. O fluxo segue para o orquestrador.

* **Condição de bloqueio:** Qualquer divergência ou ausência de credenciais resulta na interrupção imediata do fluxo, retornando o código HTTP 403 `Forbidden`.

1.3 Evidências

Os testes de integração validaram que requisições emitidas por usuários secundários tentando manipular simulações de terceiros são bloqueadas na camada HTTP, sem gerar processamento no banco de dados ou chamadas adicionais à infraestrutura.

2. Segurança e Sanitização de Registros de Execução

2.1 Objetivo

A `instrucaoEstrategica` (Prompt-Alma) contém as diretrizes comportamentais puras definidas pelo pesquisador para o agente. Por se tratar do elemento central que dita a racionalidade das ações no ciclo, este dado é classificado como sensível dentro do domínio privado do sistema. Vazamentos dessa instrução em logs de servidor representam uma quebra no isolamento da arquitetura.

2.2 Regra e Implementação

Para garantir a integridade e o sigilo deste parâmetro, foi desenvolvido um manipulador global de exceções (`sanitizacaoErros.ts`). Este componente atua como a última camada do ciclo de requisição-resposta do servidor Node.js/Express.

Sempre que uma falha ocorre na fase de decisão (como um `timeout` da API do provedor ou erro de conversão JSON), o erro original é interceptado antes de ser persistido em disco ou impresso no terminal. Uma rotina de conversão inspeciona o objeto do erro e aplica a máscara `[CONTEÚDO PROTEGIDO]` sobre qualquer incidência da propriedade

instrucaoEstrategica.

2.3 Evidências

Testes de injeção de erro comprovaram que mesmo exceções críticas e falhas de rastreamento de pilha (*stack trace*) são higienizadas com sucesso. O terminal do servidor e os logs de monitoramento registram o evento e o tipo da falha, preservando a capacidade de auditoria, mas mantêm o conteúdo da diretriz estratégica completamente ofuscado.

Figura 1 — Resposta da API no Postman.

A requisição GET /teste-erro retorna 500 Internal Server Error e uma mensagem genérica (Ocorreu um erro interno no servidor.), sem expor a instrução estratégica.

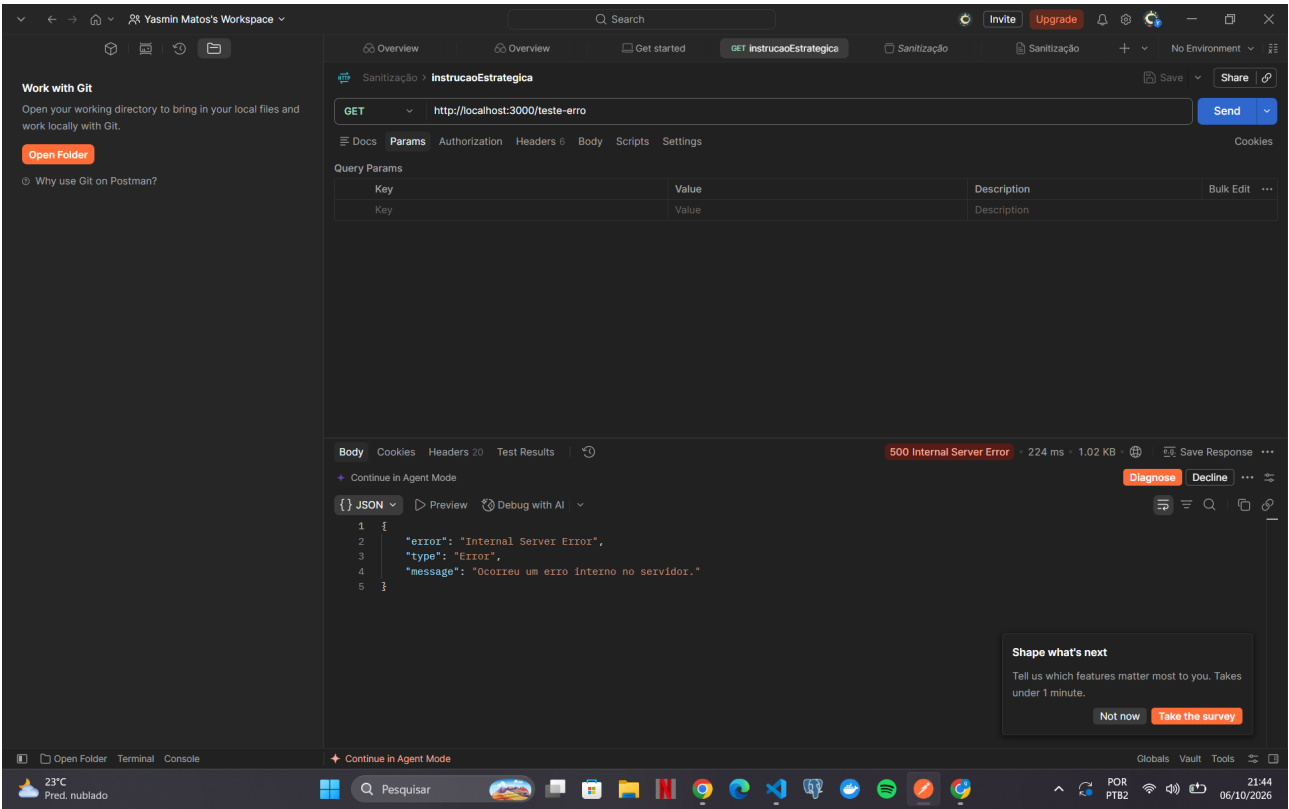
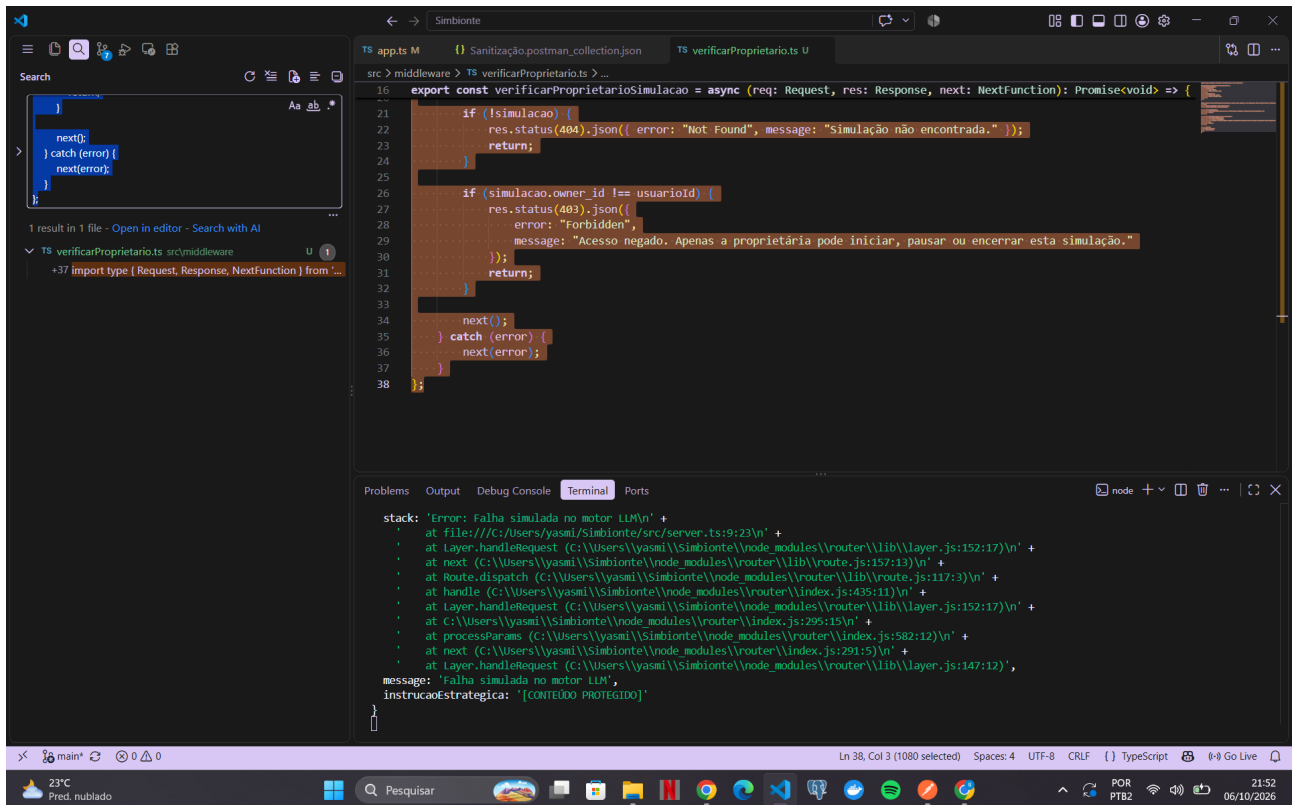


Figura 2 — Terminal do servidor após a exceção.

O *stack trace* e os dados do erro são registrados para auditoria, mas o campo `instrucaoEstrategica` aparece mascarado como `[CONTEÚDO PROTEGIDO]`.



3. Limite de Ciclos por Simulação

3.1 Objetivo

Cada ciclo de uma simulação gera chamadas pagas aos provedores de LLM (uma por agente ativo, na fase de decisão). Sem um teto, uma simulação esquecida ligada consome chamadas indefinidamente. O limite de ciclos é a trava que contém esse consumo.

3.2 Regra

1. Existe um **teto global** configurável (MAX_CICLOS_POR_SIMULACAO, padrão **100**) no .env.
2. Cada simulação pode ter um maxCiclos próprio, mas ele **nunca ultrapassa o teto global**. Se não houver valor próprio (ou for inválido: zero, negativo, decimal), vale o teto.
3. O **limite efetivo** é $\min(\text{maxCiclos da simulação}, \text{teto global})$.
4. Ao concluir o ciclo de número igual ao limite, a simulação muda para o status CONCLUIDA imediatamente, sem esperar o próximo tick.
5. Como proteção extra, antes de abrir qualquer ciclo o orquestrador confere o limite. Se uma simulação ainda EM_EXECUCAO já tiver batido o limite, ela é encerrada (CONCLUIDA) **sem executar percepção nem decisão**.
6. Simulações em qualquer status diferente de EM_EXECUCAO (pausada, concluída, aguardando) não executam ciclos.
7. Um **bloqueio por simulação** impede dois ciclos simultâneos da mesma simulação (o segundo disparo é recusado com o motivo CICLO_JA_EM_EXECUCAO). Sem isso, dois disparos sobrepostos abririam o mesmo ciclo e furariam o limite. O bloqueio é liberado mesmo se houver falha de banco no meio do ciclo.

3.3 Implementação

Peça	Arquivo	Função
Cálculo do limite efetivo	ciclo.regras.ts	calcularLimiteCiclos()
Verificação de limite	ciclo.regras.ts	limiteAtingido()
Trava no orquestrador	ciclo.orquestrador.ts	executarCiclo() (checa antes de abrir e depois de fechar o ciclo)
Configuração do teto	src/config/env.ts, .env.example	MAX_CICLOS_POR_SIMULACAO
Controle do loop com intervalo configurável (iniciar/parar)	ciclo.loop.ts	iniciarLoopDeCiclos()

3.4 Evidências

Cenários validados (todos aprovados — saída completa no Anexo A):

Cenário	Resultado esperado	Resultado
Simulação sem limite próprio	Vale o teto global (100)	Aprovado
maxCiclos (30) menor que o teto	Vale 30	Aprovado
maxCiclos (5000) maior que o teto	Vale o teto (100)	Aprovado
maxCiclos inválido (0, -3, 2.5)	Cai no teto	Aprovado
maxCiclos = 3, rodando 3 ciclos	Termina no 3º ciclo com status CONCLUIDA	Aprovado
Disparar ciclo após CONCLUIDA	Não executa e não faz nenhuma chamada ao LLM	Aprovado
Simulação EM_EXECUCAO com limite já batido	Vira CONCLUIDA sem rodar ciclo	Aprovado
Teto global menor que o maxCiclos da simulação	O teto vence	Aprovado
Simulação PAUSADA ou inexistente	Não executa ciclo	Aprovado
Dois disparos simultâneos da mesma simulação	Só um executa; o outro recebe CICLO_JA_EM_EXECUCAO	Aprovado
Falha de banco no meio do ciclo	Bloqueio liberado; próximo disparo funciona	Aprovado
Loop com intervalo de 5000 ms e limite 3	3 ciclos, 2 esperas, termina CONCLUIDA	Aprovado
parar() no meio do loop	Para sem concluir a simulação	Aprovado

Execução de demonstração (simulação com limite de 5 ciclos, 2 agentes — ver 4.5 para o agente suspenso):

```

ciclo participaram suspensos chamadas LLM status
1 [2] [1] 1 EM_EXECUCAO
2 [2] [1] 1 EM_EXECUCAO
3 [1, 2] [] 2 EM_EXECUCAO
4 [2] [1] 1 EM_EXECUCAO
5 [2] [1] 1 CONCLUIDA
tentativa 6: NÃO executou (SIMULACAO_NAO_ESTA_EM_EXECUCAO); status=CONCLUIDA
tentativa 7: NÃO executou (SIMULACAO_NAO_ESTA_EM_EXECUCAO); status=CONCLUIDA
Total de chamadas ao LLM: 6

```

Observe que as tentativas 6 e 7 **não executam** e o total de chamadas ao LLM permanece em 6.

4. Regra de Suspensão Automática de Agentes

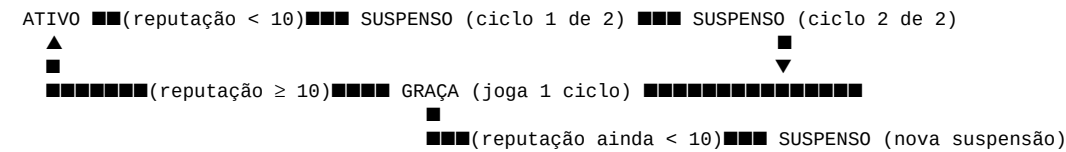
4.1 Objetivo

Agentes com reputação (TrustScore) muito baixa degradam o ecossistema. A suspensão automática os retira temporariamente da simulação e, de quebra, **evita gastar chamadas de API** com agentes que estão punidos.

4.2 Regra

- **Gatilho:** agente com reputação **abaixo de 10 pontos** (escala 0–100). Reputação exatamente igual a 10 **não** suspende.
- **Duração:** 2 ciclos.
- **Efeito:** durante a suspensão o agente **pula a fase de percepção e a fase de decisão**. Nenhum relatório é montado e **nenhuma chamada ao provedor de LLM é feita**.
- O ciclo em que a reputação baixa é detectada já conta como o **1º** ciclo suspenso.
- **Ciclo de graça (decisão de projeto):** ao terminar a suspensão, o agente joga **1 ciclo** mesmo que a reputação ainda esteja abaixo de 10, e só então é reavaliado. Sem isso, um agente que não consegue agir não teria como se recuperar e seria suspenso indefinidamente. *Esta é uma interpretação nossa — a especificação não define; confirmar com o grupo/orientação.*

4.3 Máquina de estados



Estado guardado por agente: reputacao, ciclosSuspensaoRestantes, emCicloDeGraca.

4.4 Implementação

Peça	Arquivo	Função
Regra pura (decide e devolve o novo estado)	ciclo.regras.ts	avaliarAgenteNoCiclo()
Constantes da regra	ciclo.regras.ts	REPUTACAO_MINIMA = 10, CICLOS_DE_SUSPENSAO = 2
Aplicação no orquestrador (antes de percepção/decisão)	ciclo.orquestrador.ts	executarCiclo()

A avaliação acontece **antes** de qualquer percepção/decisão, e só agentes aprovados seguem para essas fases.

4.5 Evidências

Na demonstração da seção 3.4 o agente 1 começa com reputação 5 e o agente 2 com 50:

Ciclo	Agente 1 (rep. 5)	Agente 2 (rep. 50)	Chamadas ao LLM
1	Suspenso (1º de 2)	Joga	1
2	Suspenso (2º de 2)	Joga	1
3	Joga (ciclo de graça)	Joga	2
4	Suspenso de novo (reputação seguiu < 10)	Joga	1
5	Suspenso (2º de 2) · simulação vira CONCLUIDA	Joga	1

Nos ciclos 1 e 2 o agente suspenso gerou **zero** chamadas de percepção e de decisão (verificado por contadores no teste automatizado "agente com reputação < 10 fica 2 ciclos SEM percepção e SEM decisão").

Outros cenários aprovados: reputação 10 não suspende; reputação 9,99 suspende; a suspensão dura exatamente 2 ciclos; recuperando a reputação o agente segue jogando; o estado é persistido a cada etapa; ciclo com todos os agentes suspensos ainda fecha normalmente.

4.6 Relação com a segurança dos logs

O orquestrador registra apenas identificadores (`simulacaoId`, `numeroCiclo`, `agenteId`, `evento`) e o **nome do tipo** de um eventual erro. Relatório de percepção, instrução estratégica e a mensagem do erro **não** são logados. Isso é coberto por teste: um erro simulado contendo um texto "secreto" não aparece em nenhum log gerado.

Anexo A — Saída completa dos testes automatizados

Comando: `npm run test:ciclo` (Node v22, `node --test`).

```
✓ loop: roda até o limite, espera o intervalo entre ciclos e termina CONCLUIDA
✓ loop: parar() interrompe antes do limite, sem concluir a simulação
✓ loop: simulação pausada não executa nada
✓ loop: intervalo inválido é rejeitado
✓ ciclo normal: dois agentes passam por percepção e decisão
✓ suspensão: agente com reputação < 10 fica 2 ciclos SEM percepção e SEM decisão (zero chamadas ao LLM)
✓ suspensão: o estado do agente é persistido a cada etapa
✓ suspensão: ciclo com todos os agentes suspensos ainda fecha normalmente
✓ limite: simulação com maxCiclos=3 roda 3 ciclos e vira CONCLUIDA logo no 3º
✓ limite: depois de CONCLUIDA, novos disparos não executam nada nem consomem chamadas
✓ limite: se a simulação ainda estiver EM_EXECUCAO com limite já batido, é encerrada sem rodar ciclo
✓ limite: o teto global manda mesmo se a simulação pedir mais ciclos
✓ simulação PAUSADA ou inexistente não executa ciclo
✓ falha na decisão: agente fica inativo no ciclo, ciclo fecha e o log NÃO vaza a instrução
✓ concorrência: dois disparos simultâneos não abrem o mesmo ciclo nem furam o limite
✓ falha de banco no meio do ciclo: o bloqueio é liberado e o próximo disparo funciona
✓ constantes da regra: reputação mínima 10 e suspensão de 2 ciclos
✓ limite: usa o teto global quando a simulação não define o próprio
✓ limite: o valor da simulação vale se for menor que o teto
✓ limite: o valor da simulação NUNCA passa do teto global
✓ limite: valores inválidos (0, negativo, decimal) caem no teto global
✓ limite: teto global inválido é erro de configuração
✓ limiteAtingido compara ciclos concluídos com o limite
✓ suspensão: reputação 10 (igual ao mínimo) NÃO suspende
✓ suspensão: reputação 9.99 (abaixo de 10) suspende
✓ suspensão: dura exatamente 2 ciclos e depois o agente volta (ciclo de graça)
✓ suspensão: se a reputação se recupera após a graça, o agente segue jogando
✓ avaliarAgenteNoCiclo não altera o objeto original (função pura)
# tests 28
# suites 0
# pass 28
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 418.97589
```

Anexo B — Limitações do ambiente de teste (transparência)

- Os testes usam **repositórios e LLM simulados em memória** (mesma interface que o Subgrupo A/C vão ligar). Eles validam as **regras e o fluxo do orquestrador**, mas **não** foram executados contra PostgreSQL real nem contra um provedor de LLM real, porque as tabelas de simulação/agente e o módulo de conexão ainda não existem no repositório.

- Quando o Subgrupo A liberar o serviço de início de ciclo, repetir a demonstração (`npm run demo:ciclo`) com os repositórios reais e anexar a saída aqui.